PROYECTO BIBLIOTECA IUB

Estado actualizado del proyecto

Estudiante: Isaacs Ferrer
Programa: Ingeniería Telemática
Institución: Institución Universitaria de Barranquilla (IUB)
Asignatura: [Nombre de la asignatura]
Docente: [Nombre del docente]
Fecha de elaboración: 15 de octubre de 2026

Tabla de contenido

Tabla de contenido 2
Descripción general 3
Arquitectura del sistema 3
Tecnologías utilizadas 3
Modelo de datos 4
Autenticación, autorización y roles 4
Seguridad aplicada 4
Interfaz de usuario 5
Plan de pruebas y resultados comprobados 5
Limitaciones y trabajo pendiente 5
Conclusiones 5
Referencias técnicas oficiales 6

Documento de documentación técnica del proyecto “Biblioteca IUB”. Refleja el estado actual del repositorio: Angular consume exclusivamente la API FastAPI; los módulos de Proveedores, Compras y Detalles fueron migrados a FastAPI sobre MySQL; se retiró la API auxiliar previa (Node/MySQL) del proyecto, por lo que ya no existe dependencia de ejecución hacia ella.

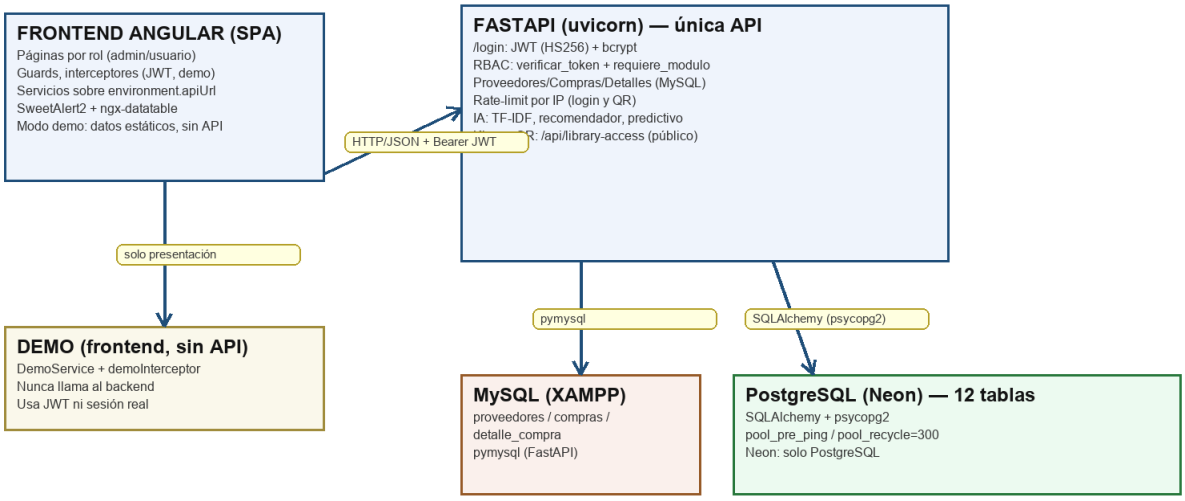
Descripción general

El sistema gestiona acceso y préstamos de recursos bibliográficos e informáticos de la Institución Universitaria de Barranquilla (IUB): autenticación por roles (JWT + RBAC), administración de libros, dispositivos y préstamos, catálogo con búsqueda tradicional y semántica, recomendaciones, panel estadístico, registro de accesos por QR, módulo de proveedores/compras y modo demostración independiente.

Arquitectura del sistema

Angular (SPA) consume una única API REST (FastAPI) con token JWT en el encabezado Authorization. La persistencia principal está en PostgreSQL (Neon); el módulo de Proveedores/Compras/Detalles usa una conexión independiente a MySQL (XAMPP) mediante pymysql, sin afectar la base principal. FastAPI aplica autenticación, autorización por módulo (RBAC) y reglas de negocio; los servicios de IA corren en el backend. El modo demostración es exclusivo del frontend e intercepta las peticiones sin contactar la API. Se retiró la API auxiliar previa (Node/MySQL); Angular ya no tiene dependencia de ejecución hacia ella.

Arquitectura de Biblioteca IUB



Tecnologías utilizadas

Tecnología	Uso en el proyecto
Angular 22 + TypeScript	Frontend (SPA, componentes standalone, señales, guards, interceptores).
Bootstrap 5 / Bootstrap Icons	Diseño responsive e identidad visual.
SweetAlert2	Alertas, confirmaciones y modales (servicio AlertaService).

Tecnología	Uso en el proyecto
ngx-datatable	Tablas de Libros y Préstamos (paginación superior, búsqueda, ordenamiento).
DataTables	Tabla de préstamos de dispositivos (paginación propia).
jsPDF / SheetJS	Exportación de reportes PDF/Excel en el cliente.
FastAPI + Uvicorn	Única API REST del backend.
SQLAlchemy + psycopg2	ORM y pool para PostgreSQL (Neon).
pymysql	Conexión independiente a MySQL (proveedores/compras).
python-jose + passlib/bcrypt	JWT (HS256) y hashing de contraseñas.
scikit-learn / pandas	Búsqueda semántica, recomendación y predicción.
PostgreSQL (Neon) / MySQL (XAMPP)	Bases de datos (principal y de compras).
Vitest + jsdom	Pruebas unitarias del frontend (@angular/build:unit-test).
Render	Hosting del backend FastAPI.

Modelo de datos

12 tablas en PostgreSQL (con el esquema documentado en biblioteca_iub_postgres.sql) más 3 tablas en MySQL para compras:

Base	Entidades
PostgreSQL (12)	rol, usuario, modulos, modulos_X_rol, libros, prestamos, dispositivos, prestamos_dispositivos, library_access, contratistas, proveer, configuracion.
MySQL (3)	proveedores, compras, detalle_compra (borrado lógico, UNIQUE y claves foráneas).

Autenticación, autorización y roles

El login valida identificación y contraseña con bcrypt y emite un JWT (HS256, 8 h) con el rol obtenido de la base de datos; el rol se revalida desde la base en cada petición protegida. En producción MM_JWT_SECRET es obligatoria (garantiza claves no predecibles). Roles: super_administrador, Administrador, Funcionario, Docente y Estudiante. Proveedores, Compras y Detalles quedan restringidos a super_administrador y Administrador (403 para los demás). El login además exige que el rol seleccionado coincida con el rol real de la cuenta.

Seguridad aplicada

Aspecto	Implementación
JWT	HS256; MM_JWT_SECRET obligatoria en Render (sin valor por defecto inseguro).
Contraseñas	bcrypt únicamente; migración explícita opcional (MM_HASH_PLANO_PERMITIDO) sin aceptar texto plano en silencio.
RBAC	Autorización por módulo (modulos_X_rol) en el backend; los controles del frontend son solo UX.

Aspecto	Implementación
CORS	Orígenes restringidos por variable de entorno (nunca comodín con credenciales).
Rate limit	Límite por IP en login y registro de accesos QR.
Pool	SQLAlchemy con pool_pre_ping y reciclado (300 s) para Neon/PgBouncer.

Interfaz de usuario

Todas las pantallas (admin y usuario) usan SweetAlert2 (servicio AlertaService) para éxito, validaciones, errores de conexión y confirmaciones, y formularios modales flotantes para el registro/edición y gestión. Las tablas de Libros, Préstamos de libros, Catálogo y Mis préstamos usan ngx-datatable con barra de paginación en la parte superior izquierda, búsqueda a la derecha, ordenamiento y selección de tamaño de página. La paginación es local y calcula las páginas a partir del total real filtrado (sin páginas vacías ni registros repetidos). La gestión de préstamos de dispositivos (admin) conserva DataTables; la solicitud de dispositivos del usuario es un modal.

Plan de pruebas y resultados comprobados

Prueba	Resultado
Backend (pytest, base SQLite aislada)	138 pruebas aprobadas (incluye CRUD de libros/dispositivos, estados Completado/Dañado, proveedores/compras sobre MySQL de pruebas, seguridad y pool).
Frontend (ng test, Vitest + jsdom)	62 pruebas aprobadas (auth, demo, interceptores, guards, login, paginación, modales).
Compilación de producción (ng build)	Exitosa.
Formato de cambios (git diff --check)	Sin errores.

Limitaciones y trabajo pendiente

#	Pendiente
1	Validar en vivo login y rutas contra el backend desplegado en Render y la base Neon.
2	Configurar en Render las variables MM_MYSQL_* para el módulo Proveedores/Compras (MySQL).
3	Publicar el frontend (dist) en su alojamiento; fijar la URL real de la API.
4	Regenerar/actualizar la documentación académica formal si cambian los requisitos del profesor.

Conclusiones

El proyecto queda consolidado con una única API (FastAPI), módulo de compras sobre MySQL, seguridad reforzada y una interfaz unificada para todos los roles. La evidencia actual: 138 pruebas backend y 62 frontend aprobadas y compilación de producción exitosa. Persisten verificaciones en vivo y el despliegue del frontend.

Referencias técnicas oficiales

Angular. https://angular.dev

Bootstrap. https://getbootstrap.com

SweetAlert2. https://sweetalert2.github.io

ngx-datatable. https://swimlane.gitbook.io/ngx-datatable

FastAPI. https://fastapi.tiangolo.com

SQLAlchemy. https://www.sqlalchemy.org

PostgreSQL. https://www.postgresql.org

MySQL. https://www.mysql.com

Neon. https://neon.tech

Render. https://render.com

scikit-learn. https://scikit-learn.org

Elaborado por: Isaacs Ferrer – Ingeniería Telemática – Institución Universitaria de Barranquilla (IUB). 15 de octubre de 2026